Sharper precision for subexponential 3SAT

A proved schedule refinement and a conditional route to a better logarithmic exponent

Research memorandum prepared for Vladimir Reshetnikov

October 26, 2026

Abstract

We analyze the October 25 revision of OpenAI's subexponential exact-counting algorithm, using a pinned source revision. We obtain a concrete refinement of its tapered Hermite interpolation: count the interpolation conditions exactly, then charge the actual two-adic loss of each recovery multiplier. The resulting working precision is at most $(6 + o(1))MD/u$, compared with the source schedule's $(24 + o(1))MD/u$. In the standard colored-partition estimate, this halves the leading constant in the exponential bound for mode enumeration. We also solve the associated real-valued minimax taper problem exactly and show why changing the taper alone cannot improve the logarithmic power.

For a stronger improvement, we give an exact compressed tensor representation of the mode sum and formulate a precise structured-summation hypothesis. Under that hypothesis, a retuned algorithm would count arbitrary fixed-width CNF in $\mathrm{poly}(n, m)2^{O(n \log\log n/\log n)}$ time and polynomial space, with $\mathrm{poly}(n, m)2^{O(n/\log n)}$ at fixed clause density. The required new summation algorithm is not proved here. Exact-arithmetic implementations check the proved precision identities and the compressed representation on finite instances.

What is established. A smaller, rigorously sufficient precision schedule; an improved explicit exponential upper-bound constant; an exact optimal taper for the full loss functional; and an exact mode-network reformulation.

What is still open in this work. An improvement of the general $O(n \log\log n/\sqrt{\log n})$ exponent to a smaller asymptotic order. The $O(n \log\log n/\log n)$ rate below is a conditional target, not a claimed new 3SAT theorem. The supplied Python programs are algebraic prototypes and schedule routines, not a full subexponential SAT solver.

Contents

1 The baseline and the result being improved **3**
1.1 A precise meaning of the proved improvement 3

2 Where the source spends exponential time **3**

3 First refinement: the exact linear taper **5**

4 Second refinement: charge the exact recovery loss **5**
4.1 Why the dominant constant becomes six 6

5 The resulting explicit exponential bound **7**

6 An exact optimal-taper theorem **8**
6.1 The finite grid and integer multiplicities 9
6.2 A reproducible finite comparison 9
6.3 What this rules out 10

7 Why ordinary mode pruning does not improve the rate **10**

8 An exact compressed representation of the mode sum **10**
8.1 Why applying the published theorem twice is invalid 11

9 The proposed improvement in logarithmic rate **12**
9.1 Two additional changes needed for arbitrary density 13

10 Research priorities and decision criteria **13**

11 Artifacts and reproducibility **14**

12 Assessment **15**

1 The baseline and the result being improved

The user's reference is OpenAI, *The Exponential Time Hypothesis is False*, dated October 17 and revised October 25, 2026 [1]. This memorandum uses repository commit `42907c9e999cc00d6abe7a50bfc1023250edd1b9`. The main theorem and quantitative appendix state, for every fixed clause width k,

$$T_{\text{general}}(n,m) = (n+m)^{O_k(1)} 2^{O_k(n \log\log n/\sqrt{\log n})}, \tag{1.1}$$

$$T_{\text{density}}(n,m) = (n+m)^{O_{k,D}(1)} 2^{O_{k,D}(n/\sqrt{\log n})} \quad (m \le Dn,\ D \text{ fixed}). \tag{1.2}$$

Both are exact-counting bounds, so testing whether the count is positive decides 3SAT when $k = 3$. Both use polynomial space.

The distinction between the manuscript and the formalized implementation matters. The repository's scope document [4] describes a Lean theorem with the weaker exponent $O(n(\log\log n)^2/\sqrt{\log n})$, and explicitly lists the sharper manuscript bounds separately. The formal solution contains a compiled counting machine [5]; it is not merely an informal algorithm sketch. We inspected that distinction but did not rebuild Lean or rerun its proof comparison machinery. The new work here is an analysis of the quantitative tensor method, not a modification of that Lean endpoint.

It would therefore be misleading to present removal of one factor of $\log\log n$ from the older bound as a new discovery. We compare against (1.1) and (1.2).

1.1 A precise meaning of the proved improvement

The change proved in Sections 3–5 improves an explicit constant in the exponent of the tensor algorithm. It leaves its big-O logarithmic rate unchanged. If two implementations are certified by $\exp((A+o(1))f(s))$ and $\exp((B+o(1))f(s))$ with $B < A$, the second has a strictly smaller stated upper bound. This does not prove that the first implementation attains its bound, or that a measured runtime ratio is $\exp((A-B)f(s))$.

Our refinement has exactly that status. The more ambitious proposal in Section 9 would change the rate itself, but depends on a new algorithmic hypothesis.

Result	Quantitative statement	Status
Exact precision schedule	$P'_t = p + B_i + (q_i - 1)(a + 1 - i)$	Proved here
Dominant precision	$24MD/u$ replaced by at most $6MD/u$, asymptotically	Proved here
Enumeration bound	$4\pi\sqrt{6}$ replaced by $2\pi\sqrt{6}$ in the normalized exponent	Proved here
Mode-network encoding	$O(M/\sqrt{u} + r^2)$ rank bits for fixed weight allocations	Exact reformulation
General 3SAT target	$\text{poly}(n,m)2^{O(n \log\log n/\log n)}$	Conditional
Fixed-density target	$\text{poly}(n,m)2^{O(n/\log n)}$	Conditional

2 Where the source spends exponential time

We use the source's contraction theorem and its correctness argument as the starting point [2, 3]. A bipartite Boolean tensor network with M wires has contraction

$$Z = \sum_{x \in \{0,1\}^E} \prod_v T_v(x|_{\delta(v)}).$$

The source handles suitably bounded degrees, polynomial-size descriptions, and ring entries in

$$(\mathbb{Z}/2^{P_0}\mathbb{Z})[V]/(V^{\ell}), \qquad P_0, \ell = O(s),$$

where s is a root size parameter and $M \le Cs$. An unramified coefficient extension supplies residue-distinct interpolation points. All precision statements below apply coordinatewise in that same extension and truncated polynomial ring.

At an active call, the construction uses packets of $L = 2^a$ slots and integer interpolation nodes

$$S = \{-L, -L+1, \dots, L-1\}, \qquad \alpha = 2^{a+1}.$$

For a retained nonzero node t, put

$$i = v_2(t), \qquad D_i = 2^{a-i}, \qquad 0 \le i < u \le a.$$

The deformed contraction $F(T)$ has degree at most

$$R = M(2L-1). \tag{2.1}$$

At each node, crossing interactions are written in mode variables Z. A mode has a positive integer weight w_Z, and its coefficient is divisible by 2^{w_Z}. There are at most $6M/D_i$ modes of any one weight. Thus only tuples satisfying

$$\sum_{Z} w_Z \beta_Z < P_t \tag{2.2}$$

can matter modulo 2^{P_t}.

For a fixed tuple, the sum factors into child contractions:

$$F(t+\alpha\zeta) = \sum_{\beta:\, \sum w_Z \beta_Z < P_t} \prod_{g=1}^{r} Z_{g,\beta} \pmod{2^{P_t}}. \tag{2.3}$$

Each mode has one source group and one target group. With the source choice $r = 64$, every child has at most $3M/32 < M/8$ wires.

The source prescribes $h_{\mathrm{old}} = \lceil 3M/u \rceil$, multiplicities $q_i = h_{\mathrm{old}}(u-i)$ for $i < u$, and

$$P_t^{\mathrm{old}} = p + 8h_{\mathrm{old}} D_i + 8M(a+1). \tag{2.4}$$

In its active regime this is $(24 + o(1))MD_i/u$. Colored partitions then give $\exp(O(M/\sqrt{u}))$ tuples. Near the root, $u = \Theta(\log s)$, producing the square root of a logarithm in the denominator.

For arbitrary clause density, the reduction first produces sparse polynomial queries. A retained incidence monomial has degree $K = O(n)$ among $N = n(\log n)^{O(1)}$ possible incidences. Binary ranks for its incidence subsets require

$$O\left(\log \binom{N+K}{K}\right) = O(n \log\log n) \tag{2.5}$$

wires. Contracting a network at that size gives (1.1). This explains the two distinct losses: the square root comes from mode enumeration, while the extra $\log\log n$ comes from the reduction's representation [3].

3 First refinement: the exact linear taper

The original multiplicity count deliberately uses a loose lower bound. The exact count immediately improves its parameter.

Lemma 3.1 (Exact number of interpolation conditions)**.** *Let $a \geq u \geq 1$, $L = 2^a$, and assign multiplicity $q_i = h(u - i)$ to every node of valuation $i < u$, and zero otherwise. The total number of Hermite conditions is*

$$Q = hA_{a,u}, \qquad A_{a,u} = L\left(2u - 2 + 2^{1-u}\right). \tag{3.1}$$

In particular $A_{a,u}$ is an integer.

Proof. For $0 \leq i < a$, exactly $L/2^i$ nonzero nodes in S have valuation i. Since $i < u \leq a$, the required sum is

$$Q = Lh \sum_{i=0}^{u-1} (u - i)2^{-i}.$$

The finite geometric sum and its derivative give $\sum_{i=0}^{u-1}(u - i)2^{-i} = 2u - 2 + 2^{1-u}$. □

Theorem 3.2 (Minimal sufficient integer slope)**.** *For the degree bound (2.1), the smallest positive integer slope within this linear-taper family that guarantees ordinary Hermite recovery is*

$$\boxed{h_{\min} = \left\lceil \frac{M(2L - 1) + 1}{L(2u - 2 + 2^{1-u})} \right\rceil}. \tag{3.2}$$

If $u \to \infty$ and $M/u \to \infty$ in the source's active regime, then $h_{\min} = (1 + o(1))M/u$.

Proof. A polynomial of degree at most R is uniquely determined by $R + 1$ Hermite conditions at distinct nodes. Equation (3.1) therefore requires $hA_{a,u} \geq R + 1$, giving (3.2) and its minimality. The denominator is $2L(u - 1 + 2^{-u})$. Dividing the numerator by this expression, and accounting for an additive rounding error below one, gives the stated asymptotic formula. □

All retained valuation levels remain active. The packets, group labels, interpolation nodes, local mode expansion, and child topology therefore need no change. If $M \geq u$, the odd nodes alone imply $A_{a,u} \geq Lu$, and $R + 1 \leq 2ML$. Hence $h_{\min} \leq \lceil 2M/u \rceil \leq 3M/u$ and $q_i \leq 3M$. These bounds are at least as strong as the auxiliary $h \leq 4M/u$, $q_i \leq 4M$ estimates used in the source.

This first change reduces the slope by a factor tending to three. The second change avoids unnecessary slack in the precision charged to each retained node.

4 Second refinement: charge the exact recovery loss

Extend q_i by zero for $i \geq u$. For a node of valuation $i < u$, define the nonnegative integer

$$B_i = \sum_{j=1}^{a-i} j\, 2^{a-i-j}(q_i - q_{i+j}) = D_i \sum_{j=1}^{a-i} \frac{j}{2^j}(q_i - q_{i+j}). \tag{4.1}$$

Nonnegativity holds for a nonincreasing taper, including both the linear taper and the rounded full-loss optimum discussed below.

For clarity, introduce the Hermite cardinal factor

$$E_t(T) = \prod_{v \in S \setminus \{0,t\}} \left(\frac{T - v}{t - v}\right)^{q_{v_2(v)}}.$$

Zero multiplicities contribute a factor one. Counting the valuations of differences in the complete residue grid gives the exact source identity

$$v_2(E_t(0)) = -B_i. \tag{4.2}$$

This is the equality preceding the source's coarse bound on Hermite loss; our change uses that equality directly [2].

Theorem 4.1 (Exact sufficient working precision)**.** *Suppose* $\deg F \le R$ *and* $F(0)$ *is integral in the source ring. At every retained node* t*, suppose* $F(t+\alpha z)$ *is integral coefficientwise and its first* q_i *coefficients are supplied modulo* $2^{P'_t}$*, where*

$$\boxed{P'_t = p + B_i + (q_i - 1)(a + 1 - i).} \tag{4.3}$$

For a nonnegative nonincreasing taper with enough Hermite conditions, these jets determine $F(0)$ *modulo* 2^p*.*

Proof. Every factor of $E_t(t+\alpha z)$ is a power of $1+\alpha z/(t-v)$. Since $v_2(t-v) \le a$, its coefficients are integral and its constant term is one. Its reciprocal truncated below q_i is therefore integral. Forming

$$G_t(z) = [F(t+\alpha z)/E_t(t+\alpha z)]_{<q_i}$$

loses no precision. Hermite recovery evaluates this polynomial at $-t/\alpha$ and multiplies by $E_t(0)$, then sums over t.

The multiplier of the coefficient of z^j, $0 \le j < q_i$, has valuation

$$v_2\Big(E_t(0)(-t/\alpha)^j\Big) = -B_i - j(a+1-i).$$

Replacing that coefficient by an arbitrary lift of its residue introduces an error with valuation at least

$$P'_t - B_i - j(a+1-i) = p + (q_i - 1 - j)(a+1-i) \ge p.$$

Every error lies in 2^p times the integral coefficient ring. Addition does not reduce this precision. The exact Hermite identity recovers $F(0)$, so the approximate calculation gives its required residue. The argument holds separately in every coefficient coordinate, including the truncated V coordinate. □

Remark 4.2. This is a sufficient precision bound. It need not be minimal for a particular polynomial with additional coefficient divisibility. For a general nonmonotone taper, a safe nondecreasing-precision version replaces the increment in (4.3) by its maximum with zero. For the positive linear taper with $u \le a$, $B_i > 0$, so precision strictly increases, preserving the source's canonical-lifting convention.

4.1 Why the dominant constant becomes six

Write $k_i = u - i$. The full normalized loss of a linear taper is

$$\sum_{j \ge 1} \frac{j}{2^j}(q_i - q_{i+j}) = h\left(6 - (k_i + 3)2^{1-k_i}\right) \le 6h. \tag{4.4}$$

The finite sum in (4.1) is no larger. Thus $B_i \le 6h_{\min}D_i$.

The source uses cumulative packet exponents

$$u_d = \left\lfloor \varepsilon\lambda/2^d \right\rfloor, \quad a_d = 2\sum_{j=0}^{d} u_j, \quad \lambda = \lceil \log_2 s \rceil, \quad \varepsilon = \frac{1}{128 \cdot 1024}.$$

At every active node, $D_i \geq 2^{a_{d-1}+u_d}$, while inherited precision is at most $s\lambda^{O(1)}2^{a_{d-1}}$. The active u_d is a positive power of λ. Consequently, uniformly over active depths and retained nodes,

$$p = o(MD_i/u), \qquad q_i(a+1-i) = o(MD_i/u).$$

The smaller new increments preserve the inherited-precision estimate. Combining these observations with Theorem 3.2 proves

$$\boxed{P'_t \leq (6+o(1))\frac{MD_i}{u}, \qquad P_t^{\mathrm{old}} = (24+o(1))\frac{MD_i}{u}.} \tag{4.5}$$

Levels close to the end of the taper can do better than the upper constant six. No change in the coefficient extension, exact arithmetic model, or polynomial-space argument is required.

5 The resulting explicit exponential bound

Let $N(P)$ be the number of mode tuples with weighted degree below P. For any $\theta > 0$, the multiplicity bound $6M/D_i$ gives

$$\log N(P) \leq \theta P + \frac{6M}{D_i}\sum_{j\geq 1} -\log(1-e^{-\theta j}) \tag{5.1}$$

$$\leq \theta P + \frac{\pi^2 M}{D_i \theta}. \tag{5.2}$$

Here and below log in an entropy calculation is natural. The second line follows by expanding the logarithm:

$$\sum_{j\geq 1} -\log(1-e^{-\theta j}) = \sum_{h\geq 1}\frac{1}{h(e^{\theta h}-1)} \leq \frac{1}{\theta}\sum_{h\geq 1}\frac{1}{h^2} = \frac{\pi^2}{6\theta}.$$

Taking $\theta = \pi\sqrt{M/(D_i P)}$ yields

$$\log N(P) \leq 2\pi\sqrt{MP/D_i}. \tag{5.3}$$

Therefore (4.5) gives the explicit comparison

$$\log N(P_t^{\mathrm{old}}) \leq (4\pi\sqrt{6}+o(1))M/\sqrt{u}, \tag{5.4}$$

$$\log N(P'_t) \leq (2\pi\sqrt{6}+o(1))M/\sqrt{u}. \tag{5.5}$$

Corollary 5.1 (An explicit bound for the complete contraction)**.** *Normalize the root so it has at most s wires, and retain the source's leaf threshold $H = \left\lceil s/\left\lfloor\sqrt{\lambda}\right\rfloor\right\rceil$ and fixed ε. With description length $\mathcal{L}$, the refined algorithm satisfies*

$$T \leq (s+\mathcal{L})^{O(1)} \exp\left[\left(\frac{2\pi\sqrt{6}}{\sqrt{\varepsilon}(1-\sqrt{2}/8)} + \log 2 + o(1)\right)\frac{s}{\sqrt{\lambda}}\right]. \tag{5.6}$$

The same calculation for the original precision prescription replaces $2\pi\sqrt{6}$ by $4\pi\sqrt{6}$.

Proof. At depth d, $M_d \le s8^{-d}$ and $u_d = (1+o(1))\varepsilon\lambda 2^{-d}$ uniformly along active paths. Thus the logarithms of the branching bounds sum to at most

$$\frac{2\pi\sqrt{6}+o(1)}{\sqrt{\varepsilon}}\frac{s}{\sqrt{\lambda}}\sum_{d\ge 0}(\sqrt{2}/8)^d.$$

Polynomial factors across $O(\log\lambda)$ levels contribute only $O(\lambda\log\lambda) = o(s/\sqrt{\lambda})$ to the logarithm. Direct enumeration at a leaf contributes $(\log 2 + o(1))s/\sqrt{\lambda}$. The allowed entry cost $\exp(O(s^{9/10}))$ is also lower order. The fixed polynomial dependence on description length is retained exactly as in the source analysis. □

This is a genuine refinement of the specified algorithm and its explicit upper bound. Its asymptotic class remains $\mathrm{poly}(s+\mathcal{L})2^{O(s/\sqrt{\log s})}$. In particular, the manuscript's general and fixed-density rates in Section 1 remain the strongest rates established here. The large fixed constants and eventual parameter thresholds in the source also remain; this analysis is not a practical performance claim.

6 An exact optimal-taper theorem

The preceding improvement raises a natural question: can a more elaborate taper reduce precision by an additional factor of u? Within the loss functional used by the source, the answer is no. We can solve the relevant real optimization problem exactly.

For a nonnegative real vector $q_0,\dots,q_{u-1}$, set $q_i = 0$ for $i \ge u$, and define

$$\mathcal{A}_i(q) = \sum_{j\ge 1}\frac{j}{2^j}(q_i - q_{i+j}), \qquad W(q) = \sum_{i=0}^{u-1} 2^{-i}q_i.$$

Theorem 6.1 (Optimal full normalized loss)**.** *For $W > 0$, among nonnegative real tapers satisfying $W(q) \ge W$,*

$$\min_q \max_{0\le i<u} \mathcal{A}_i(q) = \frac{W}{u/3 + 2/9 - (2/9)4^{-u}}. \tag{6.1}$$

Writing the right side as η, an optimizer is

$$q_i = \eta r_{u-i}, \qquad r_k = \frac{k}{6} + \frac{5}{18} + \frac{2}{9}4^{-k}. \tag{6.2}$$

Every full loss of this profile equals η.

Proof. Since $\sum_{j\ge1} j2^{-j} = 2$, the constraints $\mathcal{A}_i(q) \le \eta$ are equivalent to

$$q_i \le \frac{\eta}{2} + \sum_{j\ge 1}\frac{j}{2^{j+1}}q_{i+j}.$$

Backward induction shows that the pointwise largest feasible vector saturates each inequality. After dividing by η and reversing the index, its coefficients satisfy

$$r_k = \frac{1}{2} + \sum_{j=1}^{k-1}\frac{j}{2^{j+1}}r_{k-j}.$$

Their generating function is

$$\sum_{k\geq 1} r_k z^k = \frac{z(2-z)^2}{2(1-z)^2(4-z)}.$$

Partial fractions give (6.2); substitution also verifies it directly. Summing its weighted mass yields

$$\sum_{i=0}^{u-1} 2^{-i} r_{u-i} = \frac{u}{3} + \frac{2}{9} - \frac{2}{9}\,4^{-u}.$$

The maximal mass at loss η is therefore η times this quantity. Choosing η to attain mass W proves both directions. □

Hermite completeness asks for $W = (R+1)/L = 2M + o(M)$. Thus the optimal full normalized loss is $(6 + o(1))M/u$. The simple optimized linear taper already reaches this leading constant.

6.1 The finite grid and integer multiplicities

Let $\mathcal{A}_i^{(a)}$ truncate the defining sum after $j = a - i$. Its relation to the full loss is exact:

$$\mathcal{A}_i(q) - \mathcal{A}_i^{(a)}(q) = q_i(a - i + 2)2^{-(a-i)}. \tag{6.3}$$

For $a \geq 2u$ and $q_i = O(M)$, the right side is $o(M/u)$ uniformly as $u \to \infty$. Hence, among bounded tapers in the source regime, the finite-grid minimax loss has the same leading constant six.

An exact finite-grid optimizer can also be computed by a triangular rational recurrence. Put

$$s_i = 2 - (a - i + 2)2^{-(a-i)}, \qquad r_i^{(a)} = \frac{1 + \sum_{j=1}^{u-1-i} j2^{-j} r_{i+j}^{(a)}}{s_i},$$

processing i backwards. Scale this vector to mass W. The same backward-induction proof shows optimality for the finite normalized-loss problem. This optional finite optimizer need not be used to obtain our leading-constant improvement.

For either real optimizer, rounding every coordinate upward preserves Hermite completeness. Each loss increases by less than two: its diagonal coefficient is at most two, while all off-diagonal coefficients are nonpositive. Thus integer rounding preserves the leading constant. For the rounded full profile, monotonicity follows from that of r_k; the precision formula in Theorem 4.1 applies directly.

6.2 A reproducible finite comparison

For $M = 1000$, $a = 12$, $u = 6$, and input precision $p = 1001$, the supplied exact schedule routine gives the following maximum over retained nodes:

Schedule	Maximum working precision, in bits
Published source prescription	16,489,001
Minimal complete linear taper, exact loss	4,684,588
Rounded full-loss optimum, exact loss	3,681,886
Rounded finite-grid optimum, exact loss	3,640,672

These are exact integer schedule values, not timing measurements. The example concerns the local recovery lemma, which does not require the full solver's packet-packing inequalities. At small retained ranges, the lower-order improvement from the optimal profiles can be substantial. Upward rounding of a real optimum is not claimed to solve the integer minimax problem exactly.

6.3 What this rules out

Within the retained-level architecture, a bounded taper satisfying completeness must incur normalized loss $\Omega(M/u)$ at some active level. At that level the Hermite multiplier loses $\Omega(MD_i/u)$ bits. Therefore merely changing the taper cannot replace that scale by MD_i/u^2.

This is a restriction on a particular precision accounting method. It is not a lower bound on all algorithms. A different recovery identity, additional divisibility specific to the generated polynomials, or cancellation between recovery terms could evade its hypotheses.

7 Why ordinary mode pruning does not improve the rate

Two valid refinements look promising at first. Since the expansion is a product of M affine row factors, it satisfies $\sum_Z \beta_Z \le M$. Also, the true valuation of a mode coefficient can exceed its assigned weight by a baseline shift $a - i$. Neither restriction alone changes the worst-case counting scale.

Proposition 7.1 (A feasible binary family)**.** *Consider an abstract mode family with $c = \Theta(M/D)$ colors per weight and budget $P = \Theta(MD/u)$, in a regime where $D/\sqrt{u} \to \infty$. There are parameter choices of this order for which at least $2^{\Omega(M/\sqrt{u})}$ feasible tuples remain even after imposing $\beta_Z \in \{0, 1\}$ and $\sum_Z \beta_Z \le M$.*

Proof. Choose $J = \lfloor D/(C\sqrt{u}) \rfloor$ for a sufficiently large fixed C and select all colors at weights between J and $2J$. There are $K = \Theta(cJ) = \Theta(M/\sqrt{u})$ such modes. Each of their 2^K subsets has ordinary degree at most $K \le M$ and weight at most $2JK = O(MD/u)$, below the budget after choosing C appropriately. □

In the source's regime, the uniform baseline shift is negligible compared with J; it does not remove this family. Balanced versions also satisfy per-packet ordinary-degree caps. This proves a barrier to improving an enumeration bound from those restrictions alone. It does not assert that all coefficients in every actual network are nonzero.

Changing polynomial bases offers a related constant-level improvement, but no established new rate: factorial divisibility obeys $v_2(b!) = b - s_2(b)$, which is asymptotically linear in b. It does not provide the superlinear weight growth needed to change the partition exponent. The refinement proved above extracts the available taper constant without claiming such a change.

8 An exact compressed representation of the mode sum

The stronger proposal is to exploit the structure of (2.3) instead of attempting to enumerate fewer abstract tuples. Its source–target incidence suggests an exact second tensor network.

Partition the modes by ordered source–target pair (g, h). Let $\mathcal{Z}_{gh}$ be one such class. For an integer weight w, define

$$\mathcal{C}_{gh}(w) = \{(\beta_Z)_{Z \in \mathcal{Z}_{gh}} : \sum_{Z \in \mathcal{Z}_{gh}} w_Z \beta_Z = w\}, \qquad c_{gh}(w) = |\mathcal{C}_{gh}(w)|.$$

Enumerate arrays (w_{gh}) with total below P_t, skipping arrays whose classes are empty. For a fixed array, encode each class tuple by its lexicographic rank, using

$$b_{gh} = \lceil \log_2 c_{gh}(w_{gh}) \rceil$$

bits, with zero bits when the count is one. Unused binary rank strings give tensor entry zero.

The tensor at group g decodes the ranks of all incident mode classes and takes the value $Z_{g,\beta}$ from the source's child construction. Each tuple is visible to precisely its source and target. Summing over the rank bits therefore gives exactly the contribution of that weight array. Summing over arrays gives (2.3).

Proposition 8.1 (Size and exactness of the rank encoding)**.** *For a fixed feasible array, the number of rank bits is*

$$S_\beta \le \log_2 N(P_t) + r(r-1) = O(M/\sqrt{u} + r^2). \tag{8.1}$$

The encoding is counting preserving. Completion counts and rank decoding use polynomial time and space in the original root parameters. The array enumeration has at most $P_t^{O(r^2)}$ possibilities.

Proof. For the chosen array, its tuple count is the product of its class counts. It is at most the total feasible tuple count $N(P_t)$. Taking logarithms and allowing one rounding bit for each ordered pair proves (8.1). Class decoding is bijective on valid ranks, and invalid ranks contribute zero, so the contraction neither drops nor duplicates a tuple.

For modes with weights $v_1, \dots, v_b$, suffix completion counts satisfy

$$C(j, w) = \sum_{e=0}^{\lfloor w/v_j \rfloor} C(j+1, w - ev_j),$$

with terminal counts $C(b+1, 0) = 1$ and zero at other weights. These exact counts support ranking and unranking by subtraction of successive completion blocks. All mode counts and precision parameters are polynomial in s, and their count bit lengths are polynomial as well. The number of weight coordinates is at most $r(r-1)$, proving the array bound. □

The new graph may be made bipartite by subdividing every rank wire with an equality tensor, at a factor-two edge cost. This is an exact reformulation, not yet a faster contraction algorithm.

8.1 Why applying the published theorem twice is invalid

The compressed graph has only r genuine group tensors. In a balanced family their degrees are of order S_β/r. If r is fixed or polylogarithmic, this exceeds the published theorem's $S_\beta^{9/10}$ maximum-degree requirement. Fast entry evaluation does not remove that requirement.

The packing inequality makes the obstruction quantitative. Its array parameter is $D_{\mathrm{arr}} = r_{\mathrm{label}}^{2a}$, and it requires

$$M_{\mathrm{call}} \ge 2D_{\mathrm{arr}}^2 \Delta.$$

For outer degree $\Delta \asymp S_\beta/r$, it allows only $u' = O(\log r)$ retained levels. Increasing the number of initial groups to r simultaneously restricts the initial range to $u = O(\log M/\log r)$. The apparent two savings then cancel:

$$\frac{M}{\sqrt{\log M/\log r}} \frac{1}{\sqrt{\log r}} = \frac{M}{\sqrt{\log M}}.$$

This is a diagnosis of this particular attempted proof, not a general algorithmic lower bound.

An equality tensor can be split into a bounded-degree tree because it has special structure. A generic high-degree tensor cannot be split that way merely from an efficient procedure for querying its entries. Retaining the original child networks does supply structure, but restores $\Theta(M)$ original wires and loses the compression. A successful new theorem must use the actual generated representation more effectively.

9 The proposed improvement in logarithmic rate

The exact target can now be stated without hiding the missing step.

Hypothesis 9.1 (Structured-mode acceleration)**.** For the actual packet tensors and canonical descriptions generated by the source, there is a uniform deterministic procedure evaluating (2.3) using at most

$$s^{O(1)} \exp(O(M/u))$$

recursive contractions of the same admissible type, each with at most $M/8$ wires. It preserves the source's degree and precision bounds, polynomial description lengths, and polynomial space. Its other work has only the source's allowed polynomial factors and entry-evaluation factor $\exp(O(s^{9/10}))$. The recursive query count is independent of the original description length; that length enters as a fixed polynomial per-call work factor.

This is an algorithmic hypothesis about structured expressions, not about arbitrary unrelated summands supplied through a black-box oracle. The explicit mode enumeration in the paper does not meet it. The rank representation above is a possible starting point, but does not prove it.

Theorem 9.2 (Conditional faster contraction)**.** *Under Hypothesis 9.1, the source's class of tensor networks can be contracted in*

$$(s + \mathcal{L})^{O(1)} 2^{O(s/\log s)}$$

time and polynomial space.

Proof. Replace the leaf threshold by $H = \lceil s/\lambda \rceil$, retaining $u_d = \left\lfloor \varepsilon\lambda/2^d \right\rfloor$ and $a_d = 2\sum_{j\le d} u_j$. At an active call,

$$s/\lambda < M \le Cs8^{-d}, \qquad d \le \tfrac{1}{3}\log_2 \lambda + O_C(1), \qquad u_d = \Omega_C(\lambda^{2/3}).$$

Thus one may replace the source's auxiliary lower bound $u \ge \lambda^{4/5}$ by $u \ge \lambda^{3/5}$, for sufficiently large s. The cumulative bound $a_d \le 4\varepsilon\lambda$ is unchanged.

In particular, $L, D_{\mathrm{arr}} \le s^{1/1000}$ still holds, and

$$2D_{\mathrm{arr}}^2\Delta = O(s^{9/10+2/1000}) = o(s/\lambda), \qquad LD_{\mathrm{arr}} = o(s/\lambda), \qquad 2L \le \Delta.$$

All packing conditions survive. The inherited precision is still $s\lambda^{O(1)}2^{a_{d-1}}$, while $D_i \ge 2^{a_{d-1}+u_d}$. Since 2^{u_d} dominates every fixed power of λ, the existing $P_t = O(MD_i/u_d)$ bound survives as well.

Under the hypothesis, the logarithmic branching contribution at depth d is $O(M/u_d) = O((s/\lambda)4^{-d})$. Summing the geometric series gives $O(s/\lambda)$. Polynomial overhead across all depths contributes only $O(\lambda \log \lambda)$. A leaf costs

$$\exp(O(H) + O(s^{9/10})) = \exp(O(s/\lambda)).$$

The assumed streaming evaluation and immutable polynomial descriptions permit the same depth-first polynomial-space traversal. □

Corollary 9.3 (Conditional fixed-density counting)**.** *For fixed k and D, Hypothesis 9.1 implies exact counting of formulas with $m \le Dn$ in*

$$(n + m)^{O_{k,D}(1)} 2^{O_{k,D}(n/\log n)}$$

time and polynomial space, using the source's $O_{k,D}(n)$-wire clause network.

9.1 Two additional changes needed for arbitrary density

Changing only the contraction exponent is insufficient. The original general-density proof contains two separate factors $2^{O(n/\sqrt{\log n})}$, which would exceed the proposed faster target. Both must be retuned.

Let $\lambda = \lceil \log_2 n \rceil$ and $\nu = \lceil \log_2 \lambda \rceil$. First set the sparsification parameter to

$$\sigma = 1/\lambda. \tag{9.1}$$

The source's disjoint sparsification lemma gives $\mathrm{poly}(n, \lambda)2^{n/\lambda}$ tree nodes and occurrence bound

$$B = O_k((\lambda \log(2\lambda))^{k-1}) = \lambda^{O_k(1)}.$$

The initial parity checks, greedy candidate lengths, and sparse polynomial queries therefore still have polylogarithmic sparsity.

Second, if N_0 is the number of test–variable incidences in a query, use the high-incidence cutoff

$$D_* = \max\{1, \lceil N_0/n \rceil\}\lambda. \tag{9.2}$$

At most n/λ variables exceed it. Enumerating their assignments costs at most $2^{n/\lambda}$. The remaining scopes and degrees still carry only fixed polylogarithmic factors.

The incidence-monomial encoding continues to have $s = n\nu$ as a valid size parameter. Its maximum degree is at most $n^{3/4}\lambda^{O_k(1)} < (n\nu)^{9/10}$ for sufficiently large n, and its entry cost remains $\exp(\lambda^{O_k(1)})$. Theorem 9.2 would give exponent $O(s/\log s) = O(n\nu/\lambda)$. The two new branching exponents $O(n/\lambda)$ and the degree-array exponent $O(\sqrt{n}\log n)$ are absorbed. Candidate enumeration contributes only fixed powers of λ to the exponent.

Corollary 9.4 (Conditional general-density counting)**.** *Under Hypothesis 9.1, for every fixed k one obtains*

$$\boxed{(n+m)^{O_k(1)}2^{O_k(n \log\log n/\log n)}} \tag{9.3}$$

time and polynomial space for exact counting. In particular, this would give the same bound for deciding 3SAT.

The argument applies to every queried sparse system, including candidates that still admit assignments violating the original formula. No unproved assumption about the greedy procedure's intermediate outputs is needed.

10 Research priorities and decision criteria

The immediate implementation task is well defined: use (3.2), (4.1), and (4.3) in a quantitative contraction implementation, retaining the existing algebraic interfaces and canonical-lifting rules. The supplied schedule module provides exact integer and rational routines for this part. A future Lean formalization should first prove these local identities, then transport the source contraction correctness argument; it should not simply edit the unrelated older formalized complexity endpoint.

For a new logarithmic rate, the most focused next task is a factorization theorem for the generated high-degree tensors in Section 8. A candidate must do all of the following:

1. Preserve the exact weighted budget and the coefficient extraction at every target group, including polynomial factors in the truncated ring and creation-time representatives.

2. Use the tensor descriptions or their source–target structure, so that the exponentially many feasible tuples need not be queried separately.

3. Bound total time for computing the factorization, not just its number of rank bits. Recomputing child entries can dominate a small representation.

4. Avoid reintroducing $\Theta(M)$ uncompressed wires or violating the degree hypothesis at the next contraction level.

5. Give a polynomial-space streaming evaluation and a uniform bound valid at all active recursion depths.

A representation satisfying these conditions with the query bound in Hypothesis 9.1 would be a substantive breakthrough. Merely showing that the sum has a smaller entropy encoding does not meet them. The finite prototype is useful for falsifying proposed identities early; it cannot establish the required asymptotic representation theorem.

The taper optimum also supplies a practical stopping rule for this research direction. Further optimization of the same retained-level loss can improve lower-order terms and constants, but cannot generate an extra factor $1/\sqrt{\log s}$ in the exponent. That gain must enter through new algebraic summation, a different recovery scheme, or additional structure not captured by the present loss functional.

11 Artifacts and reproducibility

The archive contains this LaTeX source and its compiled PDF; exact-arithmetic Python routines; generated verification records; source provenance; and supporting derivations. The Python checks use the standard library.

`code/precision_schedule.py`
: Computes source, optimized-linear, and rounded optimal schedules using integers and exact rational arithmetic. It exposes the proved local refinement without claiming to execute the entire SAT algorithm.

`code/verify_precision_schedule.py`
: Checks schedule completeness, exact loss accounting, and finite recovery properties; its JSON result records the executed tests.

`code/mode_network.py`
: Implements a small exact Cauchy deformation, mode-polynomial coefficient extraction, completion-count ranking, and the compressed mode network.

`code/verify_mode_network.py`
: Compares the direct X, Y sum, global mode enumeration, and pairwise-rank contraction. Its fixtures include signed interpolation nodes, zero and nonzero shifts, invalid rank codes, and a nonzero quadratic mode term.

`code/verify_bounds.py`
: Independently checks finite Hermite valuations by rational products, finite-tail identities, taper optima, and feasible-family witnesses.

`results/`
: Contains machine-readable verification summaries and exponent-scale comparisons. The

latter compare mathematical functions, not runtime measurements. The faster rate is labeled conditional.

`notes/` and `source_manifest.json`
Contain detailed derivations, the conditional theorem's assumptions, and pinned source identities. No unproved acceleration routine is supplied under an implementation label.

The mode-representation verification includes 11 exact identity fixtures, 133,376 direct assignment pairs, 847 feasible global mode tuples, and 456 invalid binary rank strings assigned zero. All three evaluations agree. Nine accompanying unit tests pass. These are checks of the reformulation on finite examples, not a new verification of the full source paper or its asymptotic theorem.

The independent bounds verifier includes 5,460 exhaustive small taper profiles, 92,840 finite-tail coordinate checks, and 46 direct rational Hermite multiplier checks. The JSON summaries give the full parameters and exact results; subsequent reruns should be compared with those records rather than with wall-clock timing.

The precision module passes seven unit tests and 40 exact Hermite recovery fixtures, with 304 independent multiplier-valuation checks and 304 precision-boundary checks. The fixtures include genuinely rational Cauchy-product polynomials as well as integer polynomials. Errors at the permitted precision preserve the recovered residue; decreasing the allowance by one bit can change it, as checked by full rational recovery.

To reproduce the checks and rebuild the report from the archive root:

```
python3 code/verify_precision_schedule.py
python3 code/verify_mode_network.py
python3 code/verify_bounds.py
latexmk -pdf -interaction=nonstopmode -halt-on-error report.tex
```

12 Assessment

The established contribution is a concrete improvement to the quantitative implementation's schedule: the exact interpolation condition count and exact valuation charge reduce its dominant precision coefficient from 24 to at most 6, and halve the corresponding colored-partition upper-bound coefficient. The optimal-taper theorem shows that the improved leading precision coefficient is already optimal within the full retained-level loss model.

The desired improvement in logarithmic order remains a research proposal. Its missing ingredient is isolated in Hypothesis 9.1; the required recursion and both density-reduction changes have been worked out. The exact compressed representation supplies an object on which to attack that ingredient, while the degree and enumeration barriers explain why it is not a consequence of the published contraction theorem alone.

References

[1] OpenAI. *The Exponential Time Hypothesis is False*. October 17, 2026, revised October 25, 2026. Pinned paper and main LaTeX source.

[2] OpenAI. Quantitative contraction appendix to [1]. Pinned source: quantitative-contraction.tex. Especially the tapered Hermite recovery, mode multiplicity, mode budget, recursion identity, and final time analysis.

[3] OpenAI. Quantitative reduction appendix to [1]. Pinned source: quantitative-reduction.tex. Especially the Boolean contraction interface, fixed-density clause network, incidence degree, wire budget, and arbitrary-density proof.

[4] OpenAI. Formalization scope for *The Exponential Time Hypothesis is False*. Pinned scope document, lean/docs/378.md.

[5] OpenAI. Formal counting-machine endpoint. Pinned source: SqrtLogSpace/Final.lean. Inspected for implementation scope; not rebuilt in this investigation.